

CoderPad-Style Python Interview

Scripting & Automation — 100 Questions with Full Code

Questions:	100 (Debug · Fix · Implement · Optimize)
Format:	CoderPad live coding — write, run, explain
Levels:	Junior · Intermediate · Senior
Topics:	Bugs · Strings · Files · OOP · Generators Decorators · Concurrency · CLI · Testing

Table of Contents

Section 1: Debug & Fix — Core Python Bugs	Q1–Q15
Section 2: String Manipulation & Regex	Q16–Q26
Section 3: File I/O & File System Automation	Q27–Q36
Section 4: Data Structures & Algorithms	Q37–Q50
Section 5: OOP — Classes, Methods & Patterns	Q51–Q60
Section 6: Generators, Iterators & Decorators	Q61–Q68
Section 7: Error Handling & Logging	Q69–Q75
Section 8: Subprocess & OS Automation	Q76–Q82
Section 9: Concurrency & Performance	Q83–Q90
Section 10: Unit Testing & CLI Automation	Q91–Q100

Section 1: Debug & Fix — Core Python Bugs (Q1–Q15)

Q1. Fix: sum even numbers — two bugs hidden in this code.

BUG: Missing colon after if + wrong indentation on for loop

```
numbers = [1, 2, 3, 4, 5]
total = 0
for number in numbers: # IndentationError
if number % 2 == 0 # SyntaxError: missing colon
total += number
```

CORRECT: Fixed — correct indent and colon

```
numbers = [1, 2, 3, 4, 5]
total = 0
for number in numbers:
if number % 2 == 0:
total += number
print(total) # 6

# Pythonic one-liner
total = sum(n for n in numbers if n % 2 == 0)
print(total) # 6
```

Q2. Fix: remove even numbers from a list — list shrinks unexpectedly.

BUG: Mutating a list while iterating over it skips elements

```
nums = [1, 2, 3, 4, 5, 6]
for n in nums:
if n % 2 == 0:
nums.remove(n) # BUG: skips elements
print(nums) # Wrong: [1, 3, 5, 6]
```

CORRECT: Build a new list or iterate over a copy

```
nums = [1, 2, 3, 4, 5, 6]

# Fix 1: list comprehension (Pythonic)
odds = [n for n in nums if n % 2 != 0]
print(odds) # [1, 3, 5]

# Fix 2: iterate over a slice copy
for n in nums[:]:
if n % 2 == 0:
nums.remove(n)
print(nums) # [1, 3, 5]
```

Q3. Fix: Calculator class — add/subtract store to self but never return.

BUG: Methods assign self.result but callers receive None

```
class Calculator:
def add(self, a, b):
self.result = a + b # BUG: no return
def subtract(self, a, b):
self.result = a - b # BUG: no return

c = Calculator()
print(c.add(3, 4)) # None — not 7!
```

CORRECT: Return the computed value

```

class Calculator:
    def __init__(self):
        self.result = 0

    def add(self, a, b):
        self.result = a + b
        return self.result # FIX

    def subtract(self, a, b):
        self.result = a - b
        return self.result # FIX

    def divide(self, a, b):
        if b == 0:
            raise ZeroDivisionError("Cannot divide by zero")
        return a / b

c = Calculator()
print(c.add(3, 4)) # 7
print(c.subtract(10, 3)) # 7

```

Q4. Fix: default mutable argument — list grows across all calls.

BUG: Default list is created ONCE at function definition and shared

```

def add_item(item, lst=[]): # BUG: mutable default
    lst.append(item)
    return lst

print(add_item(1)) # [1]
print(add_item(2)) # [1, 2] <- unexpected!
print(add_item(3)) # [1, 2, 3] <- keeps growing!

```

CORRECT: Use None as sentinel, create list inside function

```

def add_item(item, lst=None): # FIX
    if lst is None:
        lst = []
    lst.append(item)
    return lst

print(add_item(1)) # [1]
print(add_item(2)) # [2]
print(add_item(3)) # [3]

# Same pattern for dicts and sets
def register(key, registry=None):
    if registry is None:
        registry = {}
    registry[key] = True
    return registry

```

Q5. Fix: generate_random_list runs in < 100ms — CoderPad signature perf question.

Bug: $O(n)$ list membership check inside a loop creates $O(n^2)$ total time.

BUG: list "in" check is $O(n)$, making the whole function $O(n^2)$

```

import random

def generate_random_list_slow(n):
    result = []
    while len(result) < n:

```

```

num = random.randint(0, n * 10)
if num not in result: # O(n) lookup every iteration!
    result.append(num)
return result

```

CORRECT: Use random.sample() or a set for O(1) lookups

```

import random, time

# Fix 1: random.sample — C-speed, O(n)
def generate_random_list_fast(n):
    return random.sample(range(n * 10 + 1), n)

# Fix 2: set for O(1) membership
def generate_random_list_set(n):
    seen, result = set(), []
    while len(result) < n:
        num = random.randint(0, n * 10)
        if num not in seen: # O(1)
            seen.add(num)
            result.append(num)
    return result

# Benchmark
n = 5000
t0 = time.perf_counter()
generate_random_list_fast(n)
print(f"Fast: {(time.perf_counter()-t0)*1000:.1f}ms") # ~1ms

```

Q6. Fix: Fibonacci — infinite recursion with no base case.

BUG: No base case — hits Python recursion limit immediately

```

def fib(n):
    return fib(n-1) + fib(n-2) # BUG: no base case

print(fib(5)) # RecursionError!

```

CORRECT: Add base cases; use iterative version for production

```

# Fix 1: base cases (still exponential)
def fib(n):
    if n <= 0: return 0
    if n == 1: return 1
    return fib(n-1) + fib(n-2)

# Fix 2: memoization with lru_cache — O(n)
from functools import lru_cache

@lru_cache(maxsize=None)
def fib_memo(n):
    if n <= 1: return n
    return fib_memo(n-1) + fib_memo(n-2)

# Fix 3: iterative — O(n) time, O(1) space (best)
def fib_iter(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n+1):
        a, b = b, a + b
    return b

```

```
print(fib_iter(10)) # 55
print(fib_memo(50)) # 12586269025
```

Q7. Fix: closure variable capture bug in a loop.

BUG: All lambda functions capture the same loop variable i, not its value

```
funcs = []
for i in range(5):
    funcs.append(lambda: i) # BUG: captures variable, not value

print([f() for f in funcs]) # [4, 4, 4, 4, 4] all 4!
```

CORRECT: Capture value at creation time with default argument

```
# Fix 1: default argument captures current value
funcs = []
for i in range(5):
    funcs.append(lambda x=i: x) # FIX

print([f() for f in funcs]) # [0, 1, 2, 3, 4]

# Fix 2: factory function
def make_func(n):
    return lambda: n

funcs2 = [make_func(i) for i in range(5)]
print([f() for f in funcs2]) # [0, 1, 2, 3, 4]
```

Q8. Fix: dictionary deletion during iteration — RuntimeError.

BUG: Deleting keys from a dict while iterating raises RuntimeError

```
data = {"a": 1, "b": None, "c": 3, "d": None}
for key in data:
    if data[key] is None:
        del data[key] # RuntimeError: dict changed size during iteration
```

CORRECT: Iterate over a snapshot of keys or use dict comprehension

```
data = {"a": 1, "b": None, "c": 3, "d": None}

# Fix 1: iterate over list copy of keys
for key in list(data.keys()):
    if data[key] is None:
        del data[key]
print(data) # {"a": 1, "c": 3}

# Fix 2: dict comprehension (Pythonic)
data = {"a": 1, "b": None, "c": 3, "d": None}
cleaned = {k: v for k, v in data.items() if v is not None}
print(cleaned) # {"a": 1, "c": 3}
```

Q9. Fix: UnboundLocalError on global counter variable.

BUG: Python treats counter as local due to the += assignment

```
counter = 0

def increment():
    counter += 1 # BUG: UnboundLocalError

increment() # crash!
```

CORRECT: Declare global, pass as parameter, or use a class

```
# Fix 1: global declaration
counter = 0
def increment():
    global counter
    counter += 1
increment()
print(counter) # 1

# Fix 2: return new value (functional)
def increment(c):
    return c + 1
counter = increment(counter)

# Fix 3: encapsulate in a class (best)
class Counter:
    def __init__(self): self.value = 0
    def increment(self): self.value += 1

c = Counter()
c.increment()
print(c.value) # 1
```

Q10. Fix: string concatenation in a loop is $O(n^2)$.

Strings are immutable. Each += creates a brand new string object.

BUG: $O(n^2)$ — new string object created every iteration

```
words = ["hello", "world", "from", "python"]
result = ""
for word in words:
    result += word + " " # BUG:  $O(n^2)$ 
```

CORRECT: Use str.join() for $O(n)$ string building

```
words = ["hello", "world", "from", "python"]

# Fix: str.join() —  $O(n)$ , fast
result = " ".join(words)
print(result) # hello world from python

# Benchmark
import time
n = 10000
ws = ["word"] * n

t0 = time.perf_counter()
r = ""
for w in ws: r += w
slow = (time.perf_counter()-t0)*1000

t0 = time.perf_counter()
r = "".join(ws)
fast = (time.perf_counter()-t0)*1000

print(f"Concat: {slow:.1f}ms Join: {fast:.1f}ms") # ~10x faster
```

Q11. Fix: using "is" instead of "==" for string comparison.**BUG: "is" checks identity (same object), not equality (same value)**

```

x = "hello world" # space prevents string interning
y = "hello world"
print(x is y) # False! BUG if used for equality check

# Seems to work for short strings due to interning:
a = "hello"
b = "hello"
print(a is b) # True (interned) -- unreliable!

```

CORRECT: Always use == for value equality; use "is" only for None/True/False

```

# Correct: use == for value comparison
x = input("Enter: ")
y = "hello"
if x == y: # CORRECT
    print("Match")

# "is" only for singletons:
result = None
if result is None: # CORRECT use of "is"
    print("No result")

# Identity vs equality demo:
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b) # True (same values)
print(a is b) # False (different objects)
print(a is c) # True (same object)

```

Q12. Fix: bare except swallows all errors including KeyboardInterrupt.

BUG: Bare except: catches SystemExit, KeyboardInterrupt — everything

```

def read_file(path):
    try:
        with open(path) as f:
            return f.read()
    except: # BUG: catches EVERYTHING
        return None # silently hides real errors

```

CORRECT: Catch specific exceptions; re-raise unexpected ones

```

import logging

def read_file(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        logging.warning(f"Not found: {path}")
        return None
    except PermissionError:
        logging.error(f"No permission: {path}")
        return None
    except OSError as e:
        logging.error(f"OS error: {e}")
        raise # re-raise unexpected errors

```

Q13. Fix: class-level mutable attribute shared by all instances.

BUG: Class-level list is shared by ALL instances of Dog


```

class Dog:
    tricks = [] # BUG: shared by all instances!

    def learn(self, trick):
        self.tricks.append(trick)

rex = Dog()
fido = Dog()
rex.learn("sit")
print(fido.tricks) # ["sit"] – Fido learned Rex's trick!

```

CORRECT: Create instance attribute in __init__

```

class Dog:
    def __init__(self, name):
        self.name = name
        self.tricks = [] # FIX: per-instance list

    def learn(self, trick):
        self.tricks.append(trick)

    def __repr__(self):
        return f"Dog({self.name!r}, tricks={self.tricks})"

rex = Dog("Rex")
fido = Dog("Fido")
rex.learn("sit")
rex.learn("shake")
fido.learn("roll over")
print(rex.tricks) # ["sit", "shake"]
print(fido.tricks) # ["roll over"]

```

Q14. Fix: largest number from array — naive sort gives wrong order.

CoderPad: [10, 9] should give "910" not "109". Int sort does not work.

BUG: Standard int sort differs from string-concat comparison

```

nums = [10, 9, 3, 30, 34, 5, 9]
nums.sort(reverse=True)
print("".join(map(str, nums))) # "109534330" -- WRONG

```

CORRECT: Custom comparator using string concatenation ordering

```

import functools

def largest_number(nums):
    strs = list(map(str, nums))

    def compare(a, b):
        if a + b > b + a: return -1
        if a + b < b + a: return 1
        return 0

    strs.sort(key=functools.cmp_to_key(compare))
    result = "".join(strs)
    return "0" if result[0] == "0" else result

print(largest_number([10, 9])) # "910"
print(largest_number([3, 30, 34, 5, 9])) # "9534330"
print(largest_number([0, 0])) # "0"

```

Q15. Fix: context manager file write — missing newline.

CoderPad's own example question. Code works but output lacks a newline.

BUG: No trailing newline — content merges when file is appended later

```
with open("output.txt", "w") as f:
    f.write("hello, world") # BUG: no newline
```

CORRECT: Add newline; use print() which adds newline by default

```
# Fix 1: explicit newline
with open("output.txt", "w") as f:
    f.write("hello, world\n")

# Fix 2: print() adds newline automatically
with open("output.txt", "w") as f:
    print("hello, world", file=f)

# Fix 3: write multiple lines cleanly
lines = ["line 1", "line 2", "line 3"]
with open("output.txt", "w") as f:
    f.writelines(line + "\n" for line in lines)

# Verify
with open("output.txt") as f:
    print(repr(f.read())) # "line 1\nline 2\nline 3\n"
```

Section 2: String Manipulation & Regex (Q16–Q26)

Q16. Implement: extract all integers from a string with regex.

```
import re

def extract_integers(text):
    return list(map(int, re.findall(r"-?\d+", text)))

print(extract_integers("abc 12 def -34 ghi 56")) # [12, -34, 56]
print(extract_integers("2024-03-15")) # [2024, 3, 15]
print(extract_integers("no numbers")) # []

# Extract floats too
def extract_numbers(text):
    return [float(n) for n in re.findall(r"-?\d+\.\d*", text)]

print(extract_numbers("price: 3.14, qty: 5")) # [3.14, 5.0]
```

Q17. Implement: validate email addresses with regex.

```
import re

EMAIL_RE = re.compile(
    r"^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$"
)

def is_valid_email(email):
    return bool(EMAIL_RE.match(email.strip()))

tests = [
    ("user@example.com", True),
    ("user.name+tag@co.uk", True),
    ("bad.email", False),
    ("@no-user.com", False),
    ("no-at-sign", False),
]

for email, expected in tests:
    ok = is_valid_email(email)
    print(("PASS" if ok==expected else "FAIL") + ": " + email)
```

Q18. Implement: find and replace with regex backreferences.

```
import re

# Reformat date YYYY-MM-DD -> DD/MM/YYYY
def reformat_date(text):
    return re.sub(r"(\d{4})-(\d{2})-(\d{2})", r"\3/\2/\1", text)

print(reformat_date("Born on 1990-07-15")) # Born on 15/07/1990

# Censor phone numbers
def censor_phone(text):
    return re.sub(r"\b(\d{3})[-.\s]?(\d{3})[-.\s]?(\d{4})\b",
        r"(\1) ***-****", text)

print(censor_phone("Call 555-867-5309")) # Call (555) ***-****
```

```
# Strip HTML tags
def strip_html(text):
    return re.sub(r"<[^>]+>", "", text)

print(strip_html("<b>Hello</b> World")) # Hello World
```

Q19. Implement: longest common prefix (CoderPad Q7).

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    shortest = min(strs, key=len)
    for i, ch in enumerate(shortest):
        if any(s[i] != ch for s in strs):
            return shortest[:i]
    return shortest

print(longest_common_prefix(["sumprime", "sumnum", "sumodd"])) # "sum"
print(longest_common_prefix(["flower", "flow", "flight"])) # "fl"
print(longest_common_prefix(["dog", "racecar", "car"])) # ""

# Sort trick: only need to compare first and last
def lcp_sort(strs):
    if not strs: return ""
    strs = sorted(strs)
    a, b = strs[0], strs[-1]
    i = 0
    while i < len(a) and i < len(b) and a[i] == b[i]:
        i += 1
    return a[:i]
```

Q20. Implement: remove a character from a string (CoderPad Q8).

```
def remove_char(s, c):
    # O(n) generator approach
    return "".join(ch for ch in s if ch != c)

def remove_char_re(s, c):
    import re
    return re.sub(re.escape(c), "", s)

print(remove_char("This is Life", "f")) # "This is Lie"
print(remove_char("hello world", "l")) # "heo word"
print(remove_char("aaa", "a")) # ""

# One-liner
print("hello".replace("l", "")) # "heo"

# Case-insensitive removal
def remove_char_ci(s, c):
    return "".join(ch for ch in s if ch.lower() != c.lower())

print(remove_char_ci("Hello World", "l")) # "Heo Word"
```

Q21. Implement: count word frequencies in a text.

```
from collections import Counter
import re
```

```

def word_frequency(text):
    words = re.findall(r"\b[a-zA-Z]+\b", text.lower())
    return Counter(words)

text = "the quick brown fox jumps over the lazy dog the fox"
freq = word_frequency(text)
print(freq.most_common(3)) # [("the",3),("fox",2),...]

# Manual (no Counter)
def word_freq_manual(text):
    freq = {}
    for word in text.lower().split():
        freq[word] = freq.get(word, 0) + 1
    return sorted(freq.items(), key=lambda x: x[1], reverse=True)

print(word_freq_manual("apple banana apple")) # [("apple",2),...]

```

Q22. Implement: check if a string is a palindrome (ignore punctuation).

```

import re

def is_palindrome(s):
    cleaned = re.sub(r"[^a-zA-Z0-9]", "", s).lower()
    return cleaned == cleaned[::-1]

# Two-pointer (no slicing)
def is_palindrome_ptr(s):
    s = re.sub(r"[^a-zA-Z0-9]", "", s).lower()
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]: return False
        l += 1; r -= 1
    return True

cases = [
    ("racecar", True),
    ("A man a plan a canal Panama", True),
    ("Was it a car or a cat I saw?", True),
    ("hello", False),
]
for s, exp in cases:
    ok = "PASS" if is_palindrome(s)==exp else "FAIL"
    print(ok + ": " + s[:30])

```

Q23. Implement: anagram check for two strings.

```

from collections import Counter

def is_anagram(s1, s2):
    clean = lambda s: s.replace(" ", "").lower()
    return Counter(clean(s1)) == Counter(clean(s2))

# Sort approach
def is_anagram_sort(s1, s2):
    clean = lambda s: sorted(s.replace(" ", "").lower())
    return clean(s1) == clean(s2)

cases = [
    ("listen", "silent", True),

```

```

("triangle", "integral", True),
("hello", "world", False),
("Astronomer", "Moon starrer", True),
]
for s1, s2, exp in cases:
ok = "PASS" if is_anagram(s1,s2)==exp else "FAIL"
print(ok + ": " + s1 + "/" + s2)

```

Q24. Implement: find the longest line in a file (CoderPad Q9).

```

# CoderPad one-liner:
# print(max(open("test.txt"), key=len).rstrip())

# Production-safe streaming version
def find_longest_line(filepath):
max_len, max_line = 0, ""
try:
with open(filepath) as f:
for line in f:
line = line.rstrip("\n")
if len(line) > max_len:
max_len = len(line)
max_line = line
except FileNotFoundError:
return None, 0
return max_line, max_len

import tempfile, os
with tempfile.NamedTemporaryFile("w", suffix=".txt", delete=False) as f:
f.write("short\nthis is a much longer line\nmedium\n")
name = f.name
line, length = find_longest_line(name)
print(f"Longest ({length} chars): {line!r}")
os.unlink(name)

```

Q25. Implement: reverse words in a sentence.

```

def reverse_words(sentence):
return " ".join(sentence.split()[::-1])

def reverse_words_manual(sentence):
words = sentence.split()
l, r = 0, len(words) - 1
while l < r:
words[l], words[r] = words[r], words[l]
l += 1; r -= 1
return " ".join(words)

cases = [
("hello world", "world hello"),
(" spaces here ", "here spaces"),
("one", "one"),
]
for s, exp in cases:
result = reverse_words(s)
ok = "PASS" if result==exp else "FAIL"
print(ok + ": " + repr(s) + " -> " + repr(result))

```

Q26. Implement: convert Excel column number to letter (CoderPad Q13).

```
def col_num_to_title(n):
    result = []
    while n > 0:
        n -= 1
        result.append(chr(n % 26 + ord("A")))
        n //= 26
    return "".join(reversed(result))

def title_to_col_num(s):
    result = 0
    for ch in s:
        result = result * 26 + (ord(ch) - ord("A") + 1)
    return result

# Tests
cases = [(1, "A"), (26, "Z"), (27, "AA"), (52, "AZ"), (703, "AAA")]
for num, expected in cases:
    r = col_num_to_title(num)
    print(("PASS" if r==expected else "FAIL") + ": " + str(num) + " -> " + r)

# Reverse direction
for num, col in cases:
    r = title_to_col_num(col)
    print(("PASS" if r==num else "FAIL") + ": " + col + " -> " + str(r))
```

Section 3: File I/O & File System Automation (Q27–Q36)

Q27. Implement: read CSV and find the row with maximum value in a column.

```
import csv

def max_row_by_column(filepath, column):
    with open(filepath, newline='') as f:
        rows = list(csv.DictReader(f))
        if not rows: return None
        return max(rows, key=lambda r: float(r[column]))

import io, tempfile, os
data = "name,score\nAlice,95\nBob,70\nCarol,99\n"
with tempfile.NamedTemporaryFile("w", suffix=".csv", delete=False) as f:
    f.write(data); path = f.name
top = max_row_by_column(path, "score")
print("Top scorer:", top["name"], top["score"]) # Carol 99
os.unlink(path)
```

Q28. Implement: count file types in a directory recursively.

```
from pathlib import Path
from collections import Counter

def count_file_types(directory):
    exts = [
        (f.suffix.lower() if f.suffix else "(no ext)")
        for f in Path(directory).rglob("*") if f.is_file()
    ]
    return Counter(exts)

import tempfile
with tempfile.TemporaryDirectory() as tmp:
    for name in ["a.py", "b.py", "c.txt", "d.txt", "e.csv", "f"]:
        Path(tmp, name).write_text("x")
    sub = Path(tmp, "sub")
    sub.mkdir()
    Path(sub, "g.py").write_text("x")

for ext, n in count_file_types(tmp).most_common():
    print(f"{ext:12s}: {n}")
# .py : 3
# .txt : 2
```

Q29. Implement: find all files modified within the last N hours.

```
import time
from pathlib import Path

def files_modified_within(directory, hours=24):
    cutoff = time.time() - hours * 3600
    result = []
    for f in Path(directory).rglob("*"):
        if f.is_file() and f.stat().st_mtime > cutoff:
            result.append({
                "path": str(f),
                "size_kb": round(f.stat().st_size / 1024, 1),
```



```

"modified": time.ctime(f.stat().st_mtime)
})
result.sort(key=lambda x: x["modified"], reverse=True)
return result

recent = files_modified_within("/tmp", hours=1)
for r in recent[:3]:
    print(r["modified"], r["path"])

```

Q30. Implement: safely load JSON config with defaults.

```

import json
from pathlib import Path

DEFAULTS = {"host": "localhost", "port": 5432,
            "debug": False, "max_workers": 4}

def load_config(path, defaults=None):
    config = dict(defaults or DEFAULTS)
    p = Path(path)
    if not p.exists():
        return config
    try:
        config.update(json.loads(p.read_text(encoding="utf-8")))
    except json.JSONDecodeError as e:
        print(f"Bad JSON in {path}: {e}")
    return config

import tempfile, os
with tempfile.NamedTemporaryFile("w", suffix=".json", delete=False) as f:
    json.dump({"port": 9999, "debug": True}, f)
    name = f.name
    cfg = load_config(name)
    print(cfg["host"]) # localhost (default)
    print(cfg["port"]) # 9999 (from file)
    os.unlink(name)

```

Q31. Implement: batch rename files matching a regex pattern.

```

from pathlib import Path
import re

def batch_rename(directory, pattern, replacement, dry_run=True):
    renamed = []
    for f in Path(directory).iterdir():
        if f.is_file():
            new_name = re.sub(pattern, replacement, f.name)
            if new_name != f.name:
                new_path = f.parent / new_name
                if not dry_run:
                    f.rename(new_path)
                renamed.append((f.name, new_name))
    return renamed

import tempfile
with tempfile.TemporaryDirectory() as tmp:
    for i in range(1, 4):
        Path(tmp, f"report_2024_0{i}.txt").touch()
    changes = batch_rename(tmp, r"_2024_", "_2025_", dry_run=False)

```

```
for old, new in changes:
    print(f" {old} -> {new}")
```

Q32. Implement: read the N largest lines from a big log file.

```
import heapq

def largest_n_lines(filepath, n=10, key=len):
    # heapq.nlargest streams the file – no full load
    with open(filepath) as f:
        return heapq.nlargest(n, f, key=key)

def tail(filepath, n=10):
    # Unix tail equivalent – reads from end
    with open(filepath, "rb") as f:
        f.seek(0, 2)
        size = f.tell()
        lines_found, block = 0, []
        pos = size - 1
        while pos >= 0 and lines_found <= n:
            f.seek(pos)
            char = f.read(1)
            if char == b"\n" and pos < size - 1:
                lines_found += 1
            block.append(char)
            pos -= 1
        return b"".join(reversed(block)).decode().splitlines()[-n:]

import tempfile, os
with tempfile.NamedTemporaryFile("w", suffix=".log", delete=False) as f:
    for i in range(100):
        f.write(f"line {i} " + "x"*i + "\n")
    name = f.name
    print("Longest:", largest_n_lines(name, 1)[0][:50].strip())
    os.unlink(name)
```

Q33. Implement: write a CSV report from a list of dicts.

```
import csv, io

def write_csv_report(records, filepath=None):
    if not records: return ""
    fields = list(records[0].keys())
    if filepath:
        with open(filepath, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=fields)
            writer.writeheader()
            writer.writerows(records)
    else:
        buf = io.StringIO()
        writer = csv.DictWriter(buf, fieldnames=fields)
        writer.writeheader()
        writer.writerows(records)
        return buf.getvalue()

records = [
    {"name": "Alice", "dept": "Eng", "salary": 95000},
    {"name": "Bob", "dept": "HR", "salary": 72000},
]
```

```
print(write_csv_report(records))
# name,dept,salary
# Alice,Eng,95000
```

Q34. Implement: atomic file write — prevent partial writes on crash.

```
import os, tempfile
from pathlib import Path

def atomic_write(filepath, content, encoding="utf-8"):
    filepath = Path(filepath)
    filepath.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp_path = tempfile.mkstemp(
        dir=filepath.parent, prefix=".tmp_")
    try:
        with os.fdopen(fd, "w", encoding=encoding) as f:
            f.write(content)
            f.flush()
            os.fsync(f.fileno()) # flush to disk
            os.replace(tmp_path, filepath) # atomic rename
    except Exception:
        os.unlink(tmp_path)
        raise

atomic_write("/tmp/safe_output.txt", "Hello, safe world!\n")
print(Path("/tmp/safe_output.txt").read_text())
```

Q35. Implement: recursive directory size calculator.

```
from pathlib import Path

def dir_size(path, unit="MB"):
    total = sum(f.stat().st_size
        for f in Path(path).rglob("*") if f.is_file())
    div = {"B":1, "KB":1024, "MB":1024**2, "GB":1024**3}
    return round(total / div.get(unit, 1), 2)

def dir_tree(path, indent=0):
    p = Path(path)
    if p.is_dir():
        size = dir_size(p, "KB")
        print(" " * indent + f"[{p.name}/] {size} KB")
        for child in sorted(p.iterdir()):
            dir_tree(child, indent+1)
    else:
        kb = round(p.stat().st_size / 1024, 2)
        print(" " * indent + f"{p.name} {kb} KB")

import tempfile
with tempfile.TemporaryDirectory() as tmp:
    Path(tmp, "a.py").write_text("x" * 1000)
    Path(tmp, "b.txt").write_text("y" * 2000)
    sub = Path(tmp, "sub")
    sub.mkdir()
    Path(sub, "c.py").write_text("z" * 3000)
    dir_tree(tmp)
```

Q36. Implement: compare two directories and list differences.

```
import filecmp
from pathlib import Path

def compare_dirs(dir1, dir2):
    dc = filecmp.dircmp(dir1, dir2)
    return {
        "only_in_left": dc.left_only,
        "only_in_right": dc.right_only,
        "different": dc.diff_files,
        "same": dc.same_files,
    }

import tempfile
with (tempfile.TemporaryDirectory() as d1,
    tempfile.TemporaryDirectory() as d2):
    Path(d1, "common.txt").write_text("same")
    Path(d2, "common.txt").write_text("same")
    Path(d1, "only_left.txt").write_text("left")
    Path(d2, "only_right.txt").write_text("right")
    Path(d1, "diff.txt").write_text("v1")
    Path(d2, "diff.txt").write_text("v2")
    diff = compare_dirs(d1, d2)
    for k, v in diff.items():
        print(f" {k}: {v}")
```

Section 4: Data Structures & Algorithms (Q37–Q50)

Q37. Implement: find missing number in 0..n array (CoderPad Q4).

```
def missing_sum(nums):
    n = len(nums)
    return n*(n+1)//2 - sum(nums)

def missing_xor(nums):
    # XOR trick: avoids overflow for large n
    n = len(nums)
    result = 0
    for i in range(n+1): result ^= i
    for num in nums: result ^= num
    return result

cases = [
    ([3,0,1], 2), ([0,1], 2),
    ([9,6,4,2,3,5,7,0,1], 8), ([0], 1)
]
for nums, exp in cases:
    r1, r2 = missing_sum(nums), missing_xor(nums)
    ok = "PASS" if r1==exp and r2==exp else "FAIL"
    print(ok + ": " + str(nums) + " -> " + str(r1))
```

Q38. Implement: two-sum — return indices (LeetCode/CoderPad staple).

```
def two_sum(nums, target):
    # O(n) hash map
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

def two_sum_sorted(nums, target):
    # O(n) two-pointer for sorted array
    l, r = 0, len(nums) - 1
    while l < r:
        s = nums[l] + nums[r]
        if s == target: return [l, r]
        elif s < target: l += 1
        else: r -= 1
    return []

cases = [([2,7,11,15], 9, [0,1]), ([3,2,4], 6, [1,2]), ([3,3], 6, [0,1])]
for nums, t, exp in cases:
    r = two_sum(nums, t)
    print(("PASS" if r==exp else "FAIL") + ": " + str(nums))
```

Q39. Implement: find all pairs summing to target (CoderPad Q2).

```
def find_pairs(arr, target):
    # O(n) hash set — unique pairs
    seen, pairs = set(), set()
    for num in arr:
```

```

comp = target - num
if comp in seen:
    pairs.add(tuple(sorted((num, comp))))
seen.add(num)
return sorted(pairs)

def find_pairs_indices(arr, target):
    # Return index pairs
    seen = {}
    result = []
    for i, num in enumerate(arr):
        if (target - num) in seen:
            result.append((seen[target-num], i))
        seen[num] = i
    return result

print(find_pairs([1,2,3,4,5], 4)) # [(1,3)]
print(find_pairs([-1,0,1,2,-1,-4], 0)) # [(-1,1)]
print(find_pairs_indices([2,7,11,15], 9)) # [(0,1)]

```

Q40. Implement: balanced brackets check.

```

def is_balanced(s):
    stack = []
    mapping = {"(": ")", "{": "}", "[": "]"
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack[-1] != mapping[ch]:
                return False
            stack.pop()
    return len(stack) == 0

cases = [
    ("()", True), ("()[]{} ", True), ("()", False),
    ("([)]", False), ("{}", True), ("", True), ("(((", False)
]
for s, exp in cases:
    r = is_balanced(s)
    print(("PASS" if r==exp else "FAIL") + ": " + s)

```

Q41. Implement: word break — segment string into dictionary words (CoderPad Q3).

```

def word_break(s, word_dict):
    # DP bottom-up O(n^2 * m)
    words = set(word_dict)
    n = len(s)
    dp = [False] * (n+1)
    dp[0] = True
    for i in range(1, n+1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[n]

cases = [
    ("leetcode", ["leet", "code"], True),

```

```

("applepenapple", ["apple", "pen"], True),
("catsanddog", ["cats", "dog", "sand", "an"], False),
("batcricketcricket", ["bat", "cricket"], True),
]
for s, d, exp in cases:
    r = word_break(s, d)
    print(("PASS" if r==exp else "FAIL") + ": " + s)

```

Q42. Implement: Fibonacci — all three approaches with complexity notes.

```

# O(2^n) naive recursion – only for illustration
def fib_naive(n):
    if n <= 1: return n
    return fib_naive(n-1) + fib_naive(n-2)

# O(n) memoized
from functools import lru_cache
@lru_cache(maxsize=None)
def fib_memo(n):
    if n <= 1: return n
    return fib_memo(n-1) + fib_memo(n-2)

# O(n) iterative – O(1) space (CoderPad best answer)
def fib_iter(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n+1):
        a, b = b, a+b
    return b

seq = [0,1,1,2,3,5,8,13,21,34,55]
for i, exp in enumerate(seq):
    assert fib_iter(i) == exp
    assert fib_memo(i) == exp
print("All correct! fib(50) =", fib_iter(50))

```

Q43. Implement: primality check — efficient trial division (CoderPad Q6).

```

def is_prime(n):
    # O(sqrt(n)) trial division
    if n < 2: return False
    if n == 2: return True
    if n % 2 == 0: return False
    i = 3
    while i * i <= n:
        if n % i == 0: return False
        i += 2
    return True

def primes_up_to(limit):
    # Sieve of Eratosthenes – O(n log log n)
    sieve = [True] * (limit+1)
    sieve[0] = sieve[1] = False
    for i in range(2, int(limit**0.5)+1):
        if sieve[i]:
            for j in range(i*i, limit+1, i):
                sieve[j] = False
    return [i for i in range(2, limit+1) if sieve[i]]

```

```

for n, exp in [(0,False),(1,False),(2,True),(17,True),(100,False)]:
    ok = "PASS" if is_prime(n)==exp else "FAIL"
    print(ok + ": is_prime(" + str(n) + ")")
    print("Primes to 30:", primes_up_to(30))

```

Q44. Implement: maximum subarray sum (Kadane's algorithm).

```

def max_subarray(nums):
    # O(n) time, O(1) space
    if not nums: return 0, []
    max_sum = curr = nums[0]
    start = end = temp_start = 0
    for i in range(1, len(nums)):
        if curr + nums[i] < nums[i]:
            curr = nums[i]
            temp_start = i
        else:
            curr += nums[i]
        if curr > max_sum:
            max_sum = curr
            start, end = temp_start, i
    return max_sum, nums[start:end+1]

cases = [
    ([-2,1,-3,4,-1,2,1,-5,4], 6, [4,-1,2,1]),
    ([1], 1, [1]),
    ([-1,-2,-3], -1, [-1]),
]
for nums, exp_sum, exp_sub in cases:
    s, sub = max_subarray(nums)
    ok = "PASS" if s==exp_sum else "FAIL"
    print(ok + ": sum=" + str(s) + " sub=" + str(sub))

```

Q45. Implement: swap two numbers without a temp variable (CoderPad Q11).

```

def swap_arithmetic(x, y):
    x = x + y
    y = x - y # original x
    x = x - y # original y
    return x, y

def swap_xor(x, y):
    # XOR swap – integers only, no overflow
    if x == y: return x, y
    x ^= y; y ^= x; x ^= y
    return x, y

# Python idiom
x, y = 52, 67
x, y = y, x
print(x, y) # 67 52

for x, y in [(52,67),(0,5),(-3,7),(5,5)]:
    a, b = swap_arithmetic(x, y)
    c, d = swap_xor(x, y)
    ok = "PASS" if (a==y and b==x and c==y and d==x) else "FAIL"
    print(ok + ": swap(" + str(x) + ", " + str(y) + ") -> " + str(a) + ", " + str(b))

```

Q46. Implement: bubble sort with early-exit optimization (CoderPad Q10).


```

def bubble_sort(arr):
    arr = arr[:] # do not mutate input
    n = len(arr)
    for i in range(n-1):
        swapped = False
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break # already sorted – O(n) best case
    return arr

cases = [
    ([64,34,25,12,22,11,90], [11,12,22,25,34,64,90]),
    ([1,2,3,4,5], [1,2,3,4,5]),
    ([5,4,3,2,1], [1,2,3,4,5]),
    ([], []),
]
for arr, exp in cases:
    r = bubble_sort(arr)
    ok = "PASS" if r==exp else "FAIL"
    print(ok + ": " + str(arr) + " -> " + str(r))

```

Q47. Implement: count passing cars — O(n) prefix count (CoderPad variant).

```

def count_passing_cars(A):
    """
    Count pairs (P, Q) where P=East(0) precedes Q=West(1).
    Each East car pairs with every subsequent West car.
    """
    east_count = pairs = 0
    for car in A:
        if car == 0:
            east_count += 1
        else:
            pairs += east_count
            if pairs > 1_000_000_000:
                return -1
    return pairs

cases = [[([0,1,0,1,1],5),([0,0,0,1],3),([1,0,0,1],2),([],0)]
for A, exp in cases:
    r = count_passing_cars(A)
    print(("PASS" if r==exp else "FAIL") + ": " + str(A) + " -> " + str(r))

```

Q48. Implement: find subarrays summing to target — prefix sum (CoderPad Q12).

```

def find_subarrays(nums, target):
    # Greedy non-overlapping, O(n)
    prefix_sum = {0: -1}
    curr_sum = 0
    result = []
    last_end = -1
    for i, num in enumerate(nums):
        curr_sum += num
        diff = curr_sum - target
        if diff in prefix_sum:
            start = prefix_sum[diff] + 1

```

```

if start > last_end:
    result.append((start, i))
    last_end = i
    prefix_sum[curr_sum] = i
    return result

def count_subarrays(nums, target):
    # Count ALL subarrays (overlapping allowed)
    from collections import defaultdict
    prefix = defaultdict(int)
    prefix[0] = 1
    curr = count = 0
    for num in nums:
        curr += num
        count += prefix[curr - target]
        prefix[curr] += 1
    return count

print(find_subarrays([1,2,3,4,1,2], 3)) # [(0,1),(4,5)]
print(count_subarrays([1,2,3,4,1,2], 3)) # 3

```

Q49. Implement: largest number from array (CoderPad Q1 — full solution).

```

import functools

def largest_number(nums):
    strs = list(map(str, nums))
    def compare(a, b):
        if a+b > b+a: return -1
        if a+b < b+a: return 1
        return 0
    strs.sort(key=functools.cmp_to_key(compare))
    result = "".join(strs)
    return "0" if result[0]=="0" else result

cases = [
    ([1,61,9,0], "96110"),
    ([3,30,34,5,9], "9534330"),
    ([10,2], "210"),
    ([0,0], "0"),
    ([1], "1"),
]
for nums, exp in cases:
    r = largest_number(nums)
    ok = "PASS" if r==exp else "FAIL"
    print(ok + ": " + str(nums) + " -> " + r)

```

Q50. Implement: k-th digit from the right of an integer.

```

def kth_digit(n, k):
    # k is 1-indexed from the right
    n = abs(n)
    if k <= 0: raise ValueError("k >= 1")
    for _ in range(k-1):
        n //= 10
    return n % 10

def kth_digit_str(n, k):
    s = str(abs(n))

```

```
return int(s[-k]) if k <= len(s) else 0

cases = [(12345,1,5),(12345,2,4),(12345,5,1),(12345,6,0),(-987,2,8),(0,1,0)]
for n, k, exp in cases:
    r = kth_digit(n, k)
    ok = "PASS" if r==exp else "FAIL"
    print(ok + ": kth_digit(" + str(n) + "," + str(k) + ")=" + str(r))
```

Section 5: OOP — Classes, Methods & Patterns (Q51–Q60)

Q51. Implement: Stack class with push, pop, peek, is_empty.

```
class Stack:
    def __init__(self):
        self._data = []

    def push(self, item):
        self._data.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("Pop from empty stack")
        return self._data.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("Peek at empty stack")
        return self._data[-1]

    def is_empty(self): return len(self._data) == 0
    def __len__(self): return len(self._data)
    def __repr__(self): return f"Stack({self._data})"

s = Stack()
s.push(1); s.push(2); s.push(3)
print(s.peek()) # 3
print(s.pop()) # 3
print(len(s)) # 2
print(s.is_empty()) # False
```

Q52. Implement: Queue using two stacks — amortized $O(1)$ dequeue.

```
class QueueViaStacks:
    def __init__(self):
        self._inbox = [] # new items
        self._outbox = [] # ready to dequeue

    def enqueue(self, item):
        self._inbox.append(item)

    def dequeue(self):
        if not self._outbox:
            while self._inbox:
                self._outbox.append(self._inbox.pop())
        if not self._outbox:
            raise IndexError("Empty queue")
        return self._outbox.pop()

    def is_empty(self):
        return not self._inbox and not self._outbox

q = QueueViaStacks()
q.enqueue(1); q.enqueue(2); q.enqueue(3)
print(q.dequeue()) # 1 (FIFO)
print(q.dequeue()) # 2
q.enqueue(4)
```

```
print(q.dequeue()) # 3
print(q.dequeue()) # 4
```

Q53. Implement: `__str__`, `__repr__`, `__eq__`, `__hash__` on a `Point` class.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        return f"({self.x}, {self.y})" # user-friendly

    def __repr__(self):
        return f"Point(x={self.x!r}, y={self.y!r})" # debug

    def __eq__(self, other):
        if not isinstance(other, Point): return NotImplemented
        return self.x == other.x and self.y == other.y

    def __hash__(self):
        return hash((self.x, self.y))

    def __add__(self, other):
        return Point(self.x+other.x, self.y+other.y)

p1, p2 = Point(1,2), Point(3,4)
p3 = Point(1,2)
print(str(p1)) # (1, 2)
print(repr(p1)) # Point(x=1, y=2)
print(p1==p3) # True
print(p1+p2) # (4, 6)
print({p1,p3}) # one element (equal hash)
```

Q54. Implement: `classmethod` vs `staticmethod` vs `instance method`.

```
class Temperature:
    kelvin_offset = 273.15

    def __init__(self, celsius):
        self.celsius = celsius

    # Instance method – accesses self
    def to_fahrenheit(self):
        return self.celsius * 9/5 + 32

    # Class method – accesses cls
    @classmethod
    def from_fahrenheit(cls, f):
        return cls((f - 32) * 5 / 9)

    @classmethod
    def from_kelvin(cls, k):
        return cls(k - cls.kelvin_offset)

    # Static method – no self or cls
    @staticmethod
    def is_valid(c):
        return c >= -273.15
```

```
t = Temperature(100)
print(t.to_fahrenheit()) # 212.0
print(Temperature.from_fahrenheit(32)) # Temperature(0.0)
print(Temperature.from_kelvin(373.15)) # Temperature(100.0)
print(Temperature.is_valid(-300)) # False
```

Q55. Implement: property decorator for validated attributes.

```
class Person:
    def __init__(self, name, age):
        self.name = name # goes through setter
        self.age = age

    @property
    def name(self): return self._name

    @name.setter
    def name(self, value):
        if not isinstance(value, str) or not value.strip():
            raise ValueError("Name must be non-empty string")
        self._name = value.strip().title()

    @property
    def age(self): return self._age

    @age.setter
    def age(self, value):
        if not isinstance(value, int) or not (0 <= value <= 150):
            raise ValueError(f"Age must be 0-150, got {value!r}")
        self._age = value

    @property
    def birth_year(self): return 2024 - self._age # read-only

p = Person(" alice ", 30)
print(p.name) # Alice
print(p.birth_year) # 1994
try:
    p.age = -5
except ValueError as e:
    print(e)
```

Q56. Implement: Singleton pattern — two approaches.

```
# Approach 1: __new__ override
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self, value=None):
        if not hasattr(self, "_initialized"):
            self.value = value
            self._initialized = True

a = Singleton(42)
```

```

b = Singleton(99)
print(a is b) # True – same object
print(a.value) # 42 – not overwritten

# Approach 2: thread-safe metaclass
import threading

class SingletonMeta(type):
    _instances, _lock = {}, threading.Lock()
    def __call__(cls, *a, **kw):
        with cls._lock:
            if cls not in cls._instances:
                cls._instances[cls] = super().__call__(*a, **kw)
            return cls._instances[cls]

class Logger(metaclass=SingletonMeta):
    def __init__(self):
        self.logs = []

l1, l2 = Logger(), Logger()
l1.logs.append("hello")
print(l2.logs) # ["hello"] – same instance

```

Q57. Implement: Observer / Event Bus pattern.

```

from typing import Callable, Dict, List

class EventBus:
    def __init__(self):
        self._handlers: Dict[str, List[Callable]] = {}

    def subscribe(self, event, handler):
        self._handlers.setdefault(event, []).append(handler)
        return self

    def unsubscribe(self, event, handler):
        if event in self._handlers:
            self._handlers[event] = [
                h for h in self._handlers[event] if h != handler
            ]

    def publish(self, event, **data):
        for h in self._handlers.get(event, []):
            h(**data)

bus = EventBus()
bus.subscribe("file.created",
              lambda path, size: print(f"[FILE] {path} ({size}B)"))
bus.subscribe("file.deleted",
              lambda path, **_: print(f"[DEL] {path}"))

bus.publish("file.created", path="/tmp/data.csv", size=1024)
bus.publish("file.deleted", path="/tmp/old.log")

```

Q58. Implement: dataclass with __post_init__ validation.

```

from dataclasses import dataclass, field
from typing import List
import re

```

```

@dataclass
class Employee:
    name: str
    email: str
    salary: float
    skills: List[str] = field(default_factory=list)

    def __post_init__(self):
        if not self.name.strip():
            raise ValueError("Name cannot be empty")
        if not re.match(r"^[^@]+@[^@]+\.[^@]+$", self.email):
            raise ValueError(f"Invalid email: {self.email}")
        if self.salary < 0:
            raise ValueError("Salary must be non-negative")
        self.name = self.name.strip().title()
        self.email = self.email.lower()

    @property
    def is_senior(self): return self.salary > 100_000

e = Employee("alice smith", "Alice@Example.COM", 120000)
e.skills = ["Python", "Docker"]
print(e.name) # Alice Smith
print(e.email) # alice@example.com
print(e.is_senior) # True

```

Q59. Implement: abstract base class for pluggable file parsers.

```

from abc import ABC, abstractmethod
import json, csv, io

class FileParser(ABC):
    @abstractmethod
    def parse(self, content: str) -> list: ...

    @abstractmethod
    def format_name(self) -> str: ...

class JSONParser(FileParser):
    def parse(self, content): return json.loads(content)
    def format_name(self): return "JSON"

class CSVParser(FileParser):
    def parse(self, content):
        return list(csv.DictReader(io.StringIO(content)))
    def format_name(self): return "CSV"

def process(parser: FileParser, content: str):
    data = parser.parse(content)
    print(f"[{parser.format_name()}] {len(data)} records")
    return data

process(JSONParser(), ' [{"a":1},{ "b":2}] ')
process(CSVParser(), "name,age\nAlice,30\nBob,25")

```

Q60. Implement: context manager class for timed code blocks.


```

import time

class Timer:
    def __init__(self, label=""):
        self.label = label
        self.elapsed = 0.0

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, *args):
        self.elapsed = time.perf_counter() - self.start
        lbl = f"[{self.label}] " if self.label else ""
        print(f"{lbl}Elapsed: {self.elapsed*1000:.2f}ms")
        return False # do not suppress exceptions

with Timer("sort") as t:
    data = sorted(range(100000, 0, -1))
    print(f"Done in {t.elapsed:.4f}s")

# Generator-based version
from contextlib import contextmanager

@contextmanager
def timed(label=""):
    t0 = time.perf_counter()
    info = {}
    try:
        yield info
    finally:
        info["elapsed"] = time.perf_counter() - t0
        print(f"{label}: {info['elapsed']*1000:.2f}ms")

```

Section 6: Generators, Iterators & Decorators (Q61–Q68)

Q61. Implement: Fibonacci generator — infinite and finite versions.

```
def fibonacci():
    # Infinite generator
    a, b = 0, 1
    while True:
        yield a # suspend here, return a
        a, b = b, a + b # resume on next()

gen = fibonacci()
first10 = [next(gen) for _ in range(10)]
print(first10) # [0,1,1,2,3,5,8,13,21,34]

def fibonacci_n(n):
    # Finite version
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

print(list(fibonacci_n(8))) # [0,1,1,2,3,5,8,13]

# Memory comparison
import sys
gen_size = sys.getsizeof(x for x in range(1_000_000))
list_size = sys.getsizeof([x for x in range(1_000_000)])
print(f"Generator: {gen_size}B List: {list_size//1024}KB")
```

Q62. Implement: generator pipeline for large log file processing.

```
def read_lines(filename):
    with open(filename) as f:
        yield from f

def strip_lines(lines):
    for line in lines: yield line.strip()

def filter_errors(lines):
    for line in lines:
        if "ERROR" in line or "CRITICAL" in line:
            yield line

def parse_log_line(lines):
    import re
    pat = re.compile(r"(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) (\w+) (.+)")
    for line in lines:
        m = pat.match(line)
        if m:
            yield {"ts": m.group(1), "level": m.group(2), "msg": m.group(3)}

def error_pipeline(filepath):
    return parse_log_line(filter_errors(strip_lines(read_lines(filepath))))

import tempfile, os
with tempfile.NamedTemporaryFile("w", suffix=".log", delete=False) as f:
    f.write("2024-03-01 10:00:00 INFO App started\n")
```

```
f.write("2024-03-01 10:01:00 ERROR DB timeout\n")
name = f.name
for event in error_pipeline(name):
    print(event)
os.unlink(name)
```

Q63. Implement: memoize decorator from scratch with TTL support.

```
import functools, time

def memoize(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    wrapper.cache = cache
    wrapper.cache_clear = lambda: cache.clear()
    return wrapper

@memoize
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

print(fib(50)) # 12586269025 - instant

# TTL cache
def ttl_cache(ttl=60):
    def decorator(func):
        cache = {}
        @functools.wraps(func)
        def wrapper(*args):
            now = time.time()
            if args in cache:
                result, ts = cache[args]
                if now - ts < ttl: return result
            result = func(*args)
            cache[args] = (result, now)
            return result
        return wrapper
    return decorator

@ttl_cache(ttl=5)
def get_config(key):
    print(f"Fetching {key}...")
    return {"timeout": 30}.get(key)

print(get_config("timeout")) # Fetching... 30
print(get_config("timeout")) # 30 (cached)
```

Q64. Implement: retry decorator with exponential backoff.

```
import functools, time, random

def retry(max_attempts=3, base_delay=1.0, exceptions=(Exception,)):
    def decorator(func):
        @functools.wraps(func)
```

```

def wrapper(*args, **kwargs):
    delay = base_delay
    for attempt in range(1, max_attempts+1):
        try:
            return func(*args, **kwargs)
        except exceptions as e:
            if attempt == max_attempts:
                print(f"All {max_attempts} attempts failed.")
                raise
            jitter = random.uniform(0, delay*0.1)
            print(f"Attempt {attempt} failed: {e}. Retry in {delay:.1f}s")
            time.sleep(delay + jitter)
            delay *= 2
    return wrapper
return decorator

call_count = 0

@retry(max_attempts=3, base_delay=0.01, exceptions=(ValueError,))
def flaky():
    global call_count
    call_count += 1
    if call_count < 3: raise ValueError(f"fail #{call_count}")
    return "Success!"

print(flaky()) # Success!
print(call_count) # 3

```

Q65. Implement: timing and logging decorators.

```

import functools, time, logging

logging.basicConfig(level=logging.INFO, format="%(message)s")

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        ms = (time.perf_counter()-t0)*1000
        logging.info(f"{func.__name__} took {ms:.2f}ms")
        return result
    return wrapper

def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        logging.info(f"CALL {func.__name__}({args}, {kwargs})")
        try:
            result = func(*args, **kwargs)
            logging.info(f"OK {func.__name__} -> {result!r}")
            return result
        except Exception as e:
            logging.error(f"FAIL {func.__name__}: {e}")
            raise
    return wrapper

@timed
@log_calls

```

```
def fetch_user(user_id):
    time.sleep(0.05)
    return {"id": user_id, "name": "Alice"}

fetch_user(42)
```

Q66. Implement: custom iterator class for even numbers.

```
class EvenRange:
    def __init__(self, start, stop, step=2):
        self.current = start if start%2==0 else start+1
        self.stop = stop
        self.step = step

    def __iter__(self): return self

    def __next__(self):
        if self.current >= self.stop:
            raise StopIteration
        value = self.current
        self.current += self.step
        return value

    def __len__(self):
        if self.stop <= self.current: return 0
        return (self.stop - self.current + self.step - 1) // self.step

print(list(EvenRange(0, 10))) # [0,2,4,6,8]
print(list(EvenRange(1, 11))) # [2,4,6,8,10]
print(len(EvenRange(0, 100))) # 50

# Generator expression equivalent
evens = (n for n in range(0, 10) if n%2==0)
print(list(evens)) # [0,2,4,6,8]
```

Q67. Implement: class decorator to add logging to every method.

```
import functools, logging

logging.basicConfig(level=logging.INFO, format="%(message)s")

def log_all_methods(cls):
    for attr_name in dir(cls):
        if attr_name.startswith("_"): continue
        attr = getattr(cls, attr_name)
        if not callable(attr): continue
        def make_logged(fn):
            @functools.wraps(fn)
            def logged(self, *args, **kwargs):
                logging.info(f"CALL {cls.__name__}.{fn.__name__}")
                result = fn(self, *args, **kwargs)
                logging.info(f" -> {result!r}")
                return result
            return logged
        setattr(cls, attr_name, make_logged(attr))
    return cls

@log_all_methods
class DataProcessor:
```

```

def load(self, path): return f"data from {path}"
def process(self, data): return data.upper()
def save(self, data, path): return f"saved to {path}"

dp = DataProcessor()
data = dp.load("/tmp/input.csv")
out = dp.process(data)
dp.save(out, "/tmp/output.csv")

```

Q68. Implement: LRU cache from scratch using OrderedDict.

```

from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self._cache = OrderedDict()

    def get(self, key):
        if key not in self._cache: return -1
        self._cache.move_to_end(key) # mark recently used
        return self._cache[key]

    def put(self, key, value):
        if key in self._cache:
            self._cache.move_to_end(key)
            self._cache[key] = value
        if len(self._cache) > self.capacity:
            self._cache.popitem(last=False) # evict LRU

    def __repr__(self): return f"LRU({dict(self._cache)})"

cache = LRUCache(3)
cache.put(1, "a"); cache.put(2, "b"); cache.put(3, "c")
print(cache.get(1)) # "a" (1 becomes most recent)
cache.put(4, "d") # evicts 2 (LRU)
print(cache.get(2)) # -1 (evicted)
print(cache) # LRU({3:"c",1:"a",4:"d"})

```

Section 7: Error Handling & Logging (Q69–Q75)

Q69. Implement: custom exception hierarchy.

```
class AppError(Exception):
    def __init__(self, message, code=None):
        super().__init__(message)
        self.code = code

class DatabaseError(AppError): pass
class NetworkError(AppError): pass
class ValidationError(AppError): pass

def connect_db(url):
    if "bad" in url:
        raise DatabaseError("Cannot connect", code="DB_001")
    return {"url": url, "connected": True}

def fetch_user(db, user_id):
    if user_id <= 0:
        raise ValidationError("user_id must be positive")
    return {"id": user_id, "name": "Alice"}

try:
    db = connect_db("bad_url")
    user = fetch_user(db, -1)
except ValidationError as e:
    print(f"Bad input: {e}")
except DatabaseError as e:
    print(f"DB error [{e.code}]: {e}") # DB_001
except AppError as e:
    print(f"App error: {e}")
except Exception as e:
    print(f"Unexpected: {e}")
    raise # always re-raise unexpected
```

Q70. Implement: structured logging with rotating file handler.

```
import logging, sys
from typing import Optional

def get_logger(name, level=logging.INFO, filepath: Optional[str]=None):
    logger = logging.getLogger(name)
    logger.setLevel(level)
    if logger.handlers:
        return logger # avoid duplicate handlers
    fmt = logging.Formatter(
        "%(asctime)s %(levelname)-8s [%s] %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S"
    )
    ch = logging.StreamHandler(sys.stdout)
    ch.setFormatter(fmt)
    logger.addHandler(ch)
    if filepath:
        import logging.handlers
        fh = logging.handlers.RotatingFileHandler(
            filepath, maxBytes=5*1024*1024, backupCount=3)
        fh.setFormatter(fmt)
```

```

logger.addHandler(fh)
return logger

log = get_logger("myapp", logging.DEBUG)
log.info("Server ready on port 8080")
log.warning("High memory: 87%")
try:
    1/0
except ZeroDivisionError:
    log.exception("Division error") # auto includes traceback

```

Q71. Implement: context manager to suppress and log specific errors.

```

from contextlib import contextmanager
import logging

@contextmanager
def safe_run(name, suppress=(Exception,), reraise=False):
    try:
        yield
    except suppress as e:
        logging.warning(f"[{name}] Suppressed {type(e).__name__}: {e}")
    if reraise:
        raise

with safe_run("config load", suppress=(FileNotFoundError,)):
    open("/nonexistent/path.conf")

with safe_run("data process", suppress=(ValueError,)):
    int("not a number")

print("Execution continues after suppressed errors")

# Standard library version
from contextlib import suppress
with suppress(FileNotFoundError, PermissionError):
    open("/root/secret.txt")

```

Q72. Implement: Result type — avoid exceptions for control flow.

```

from dataclasses import dataclass
from typing import TypeVar, Generic, Optional
T = TypeVar("T")

@dataclass
class Result(Generic[T]):
    value: Optional[T] = None
    error: Optional[str] = None

    @property
    def ok(self): return self.error is None

    def unwrap(self):
        if not self.ok: raise RuntimeError(self.error)
        return self.value

    @classmethod
    def success(cls, v): return cls(value=v)

```



```

@classmethod
def failure(cls, e): return cls(error=e)

def safe_divide(a, b):
    if b == 0: return Result.failure("Division by zero")
    return Result.success(a / b)

def safe_int(s):
    try: return Result.success(int(s))
    except ValueError: return Result.failure(f"Cannot parse {s!r}")

r1 = safe_divide(10, 2); print(r1.ok, r1.value) # True 5.0
r2 = safe_divide(10, 0); print(r2.error) # Division by zero
r3 = safe_int("42"); print(r3.unwrap()) # 42
r4 = safe_int("abc"); print(r4.error) # Cannot parse...

```

Q73. Implement: runtime type validation decorator.

```

import functools
from typing import get_type_hints
import inspect

def validate_types(func):
    hints = {k:v for k,v in get_type_hints(func).items()
              if k != "return"}
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        sig = inspect.signature(func)
        bound = sig.bind(*args, **kwargs)
        bound.apply_defaults()
        for param, val in bound.arguments.items():
            if param in hints:
                exp = hints[param]
                if not isinstance(val, exp):
                    raise TypeError(
                        f"{param}={val!r}: expected {exp.__name__}, "
                        f"got {type(val).__name__}")
        return func(*args, **kwargs)
    return wrapper

@validate_types
def create_user(name: str, age: int, active: bool = True):
    return {"name": name, "age": age, "active": active}

print(create_user("Alice", 30)) # OK
try:
    create_user("Carol", "thirty") # TypeError
except TypeError as e:
    print(e)

```

Q74. Implement: circuit breaker pattern.

```

import time

class CircuitBreaker:
    CLOSED = "closed"; OPEN = "open"; HALF_OPEN = "half_open"

    def __init__(self, max_failures=3, reset_timeout=30):
        self.max_failures = max_failures

```

```

self.reset_timeout = reset_timeout
self.failures = 0
self.state = self.CLOSED
self.last_fail_at = None

def call(self, func, *args, **kwargs):
    if self.state == self.OPEN:
        elapsed = time.time() - self.last_fail_at
        if elapsed > self.reset_timeout:
            self.state = self.HALF_OPEN
        else:
            raise RuntimeError("Circuit OPEN – unavailable")
        try:
            result = func(*args, **kwargs)
            if self.state == self.HALF_OPEN: self._reset()
            return result
        except Exception:
            self._record_failure(); raise

    def _record_failure(self):
        self.failures += 1
        self.last_fail_at = time.time()
        if self.failures >= self.max_failures:
            self.state = self.OPEN
            print(f"Circuit OPENED after {self.failures} failures")

    def _reset(self):
        self.failures = 0; self.state = self.CLOSED
        print("Circuit CLOSED – recovered")

cb = CircuitBreaker(max_failures=2, reset_timeout=1)
def bad(): raise ConnectionError("Down")
for i in range(4):
    try: cb.call(bad)
    except Exception as e: print(f"Call {i+1}: {e}")

```

Q75. Implement: pytest-style unit tests for a CSV processor.

```

import csv, io

def process_csv(content):
    reader = csv.DictReader(io.StringIO(content))
    total = 0
    for row in reader:
        try: total += float(row["amount"])
        except (KeyError, ValueError): pass
    return total

# Tests
def test_basic():
    assert process_csv("name,amount\nAlice,100\nBob,200\n") == 300.0

def test_empty():
    assert process_csv("name,amount\n") == 0.0

def test_no_amount_col():
    assert process_csv("name,score\nAlice,99\n") == 0.0

def test_invalid_value():

```

```
assert process_csv("name,amount\nAlice,bad\nBob,50\n") == 50.0

def test_floats():
    assert abs(process_csv("name,amount\nAlice,1.5\nBob,2.75\n") - 4.25) < 1e-9

for test in [test_basic, test_empty, test_no_amount_col,
test_invalid_value, test_floats]:
    try:
        test()
        print("PASS " + test.__name__)
    except AssertionError as e:
        print("FAIL " + test.__name__ + ": " + str(e))
```

Section 8: Subprocess & OS Automation (Q76–Q82)

Q76. Implement: run shell commands safely and capture output.

```
import subprocess

def run(cmd, check=True, timeout=30, cwd=None):
    result = subprocess.run(
        cmd if isinstance(cmd, list) else cmd.split(),
        capture_output=True,
        text=True,
        check=check, # raises CalledProcessError on failure
        timeout=timeout,
        cwd=cwd
    )
    return result.stdout.strip()

try:
    out = run(["python3", "--version"])
    print(out) # Python 3.x.x

    out = run("ls /tmp")
    print(out[:80])

except subprocess.CalledProcessError as e:
    print(f"Failed (exit {e.returncode}): {e.stderr}")
except subprocess.TimeoutExpired:
    print("Command timed out")
except FileNotFoundError as e:
    print(f"Not found: {e}")
```

Q77. Implement: run multiple commands concurrently with ThreadPoolExecutor.

```
import subprocess
from concurrent.futures import ThreadPoolExecutor, as_completed

def run_cmd(cmd, timeout=10):
    try:
        r = subprocess.run(cmd, capture_output=True, text=True,
            timeout=timeout, check=True)
        return cmd, True, r.stdout.strip()
    except subprocess.CalledProcessError as e:
        return cmd, False, e.stderr.strip()
    except subprocess.TimeoutExpired:
        return cmd, False, "TIMEOUT"

commands = [
    ["python3", "-c", "print('task 1')"],
    ["python3", "-c", "import time; time.sleep(0.1); print('task 2')"],
    ["python3", "-c", "print('task 3')"],
]

with ThreadPoolExecutor(max_workers=3) as pool:
    futures = [pool.submit(run_cmd, cmd) for cmd in commands]
    for fut in as_completed(futures):
        cmd, ok, out = fut.result()
        status = "OK" if ok else "FAIL"
        print(f"[{status}] {out}")
```

Q78. Implement: system health check with disk and env var validation.

```
import os, shutil

def check_disk(path="/", min_gb=5.0):
    total, used, free = shutil.disk_usage(path)
    free_gb = free / 1024**3
    used_pct = used / total * 100
    return {
        "path": path, "free_gb": round(free_gb, 2),
        "used_pct": round(used_pct, 1), "ok": free_gb >= min_gb
    }

def check_env_vars(required):
    missing = [k for k in required if not os.environ.get(k)]
    return {"ok": not missing, "missing": missing}

def health_check():
    results = {
        "disk": check_disk("/", min_gb=1.0),
        "env": check_env_vars(["HOME", "PATH"]),
    }
    all_ok = all(v["ok"] for v in results.values())
    return all_ok, results

ok, checks = health_check()
for name, info in checks.items():
    status = "PASS" if info["ok"] else "FAIL"
    print(f"[{status}] {name}: {info}")
print("Overall:", "HEALTHY" if ok else "UNHEALTHY")
```

Q79. Implement: poll a directory for new or changed files.

```
from pathlib import Path

class DirPoller:
    def __init__(self, directory):
        self.directory = Path(directory)
        self._snapshot = {}

    def _take_snapshot(self):
        return {
            str(f): f.stat().st_mtime
            for f in self.directory.rglob("*") if f.is_file()
        }

    def poll(self):
        new_snap = self._take_snapshot()
        added = [p for p in new_snap if p not in self._snapshot]
        changed = [p for p in new_snap if p in self._snapshot
                    and new_snap[p] != self._snapshot[p]]
        deleted = [p for p in self._snapshot if p not in new_snap]
        self._snapshot = new_snap
        return added, changed, deleted

import tempfile
with tempfile.TemporaryDirectory() as tmp:
    poller = DirPoller(tmp)
    poller.poll() # init snapshot
    Path(tmp, "a.txt").write_text("hello")
```

```

added, changed, deleted = poller.poll()
print("Added:", [Path(p).name for p in added])
Path(tmp, "a.txt").write_text("changed")
added, changed, deleted = poller.poll()
print("Changed:", [Path(p).name for p in changed])

```

Q80. Implement: .env file loader into os.environ.

```

import os, re
from pathlib import Path

def load_env(filepath=".env", override=False):
    path = Path(filepath)
    if not path.exists(): return {}
    loaded = {}
    for line in path.read_text().splitlines():
        line = line.strip()
        if not line or line.startswith("#"): continue
        m = re.match(r"^[A-Za-z_][A-Za-z0-9_]*=(.*)$", line)
        if m:
            key, val = m.group(1), m.group(2).strip('"\'")
            if override or key not in os.environ:
                os.environ[key] = val
            loaded[key] = val
    return loaded

def require_env(*keys):
    missing = [k for k in keys if not os.environ.get(k)]
    if missing:
        raise EnvironmentError(f"Missing: {missing}")

import tempfile
with tempfile.NamedTemporaryFile("w", suffix=".env", delete=False) as f:
    f.write("APP_NAME=MyApp\nDEBUG=true\nPORT=8080\n")
    name = f.name
    loaded = load_env(name, override=True)
    print(loaded)
    require_env("APP_NAME", "PORT") # OK
    os.unlink(name)

```

Q81. Implement: idempotent project directory scaffold.

```

from pathlib import Path
import logging

logging.basicConfig(level=logging.INFO, format="%(message)s")

STRUCTURE = {
    "src": {"__init__.py": "", "main.py": "print(42)\n"},
    "tests": {"__init__.py": "", "test_main.py": ""},
    "docs": {},
    ".gitignore": "__pycache__/\n*.pyc\n.env\n",
    "README.md": "# Project\n",
}

def scaffold(base, structure):
    base = Path(base)
    base.mkdir(parents=True, exist_ok=True)
    for name, content in structure.items():

```

```

path = base / name
if isinstance(content, dict):
if not path.exists():
logging.info(f" mkdir {path.name}/")
path.mkdir(exist_ok=True)
scaffold(path, content)
else:
if not path.exists():
logging.info(f" create {path.name}")
path.write_text(content)
else:
logging.info(f" skip {path.name} (exists)")

import tempfile
with tempfile.TemporaryDirectory() as tmp:
scaffold(tmp + "/myproject", STRUCTURE)
scaffold(tmp + "/myproject", STRUCTURE) # idempotent – safe to run twice

```

Q82. Implement: argparse CLI that filters and writes CSV.

```

import argparse, csv, sys
from pathlib import Path

def filter_csv(input_path, output_path, column, value, op="eq"):
with open(input_path) as fin, open(output_path, "w", newline="") as fout:
reader = csv.DictReader(fin)
writer = csv.DictWriter(fout, fieldnames=reader.fieldnames)
writer.writeheader()
kept = 0
for row in reader:
cell = row.get(column, "")
match = False
if op == "eq": match = cell == value
elif op == "gt": match = float(cell) > float(value)
elif op == "contains": match = value in cell
if match:
writer.writerow(row); kept += 1
return kept

# Test in-process
import tempfile, os
data = "name,score\nAlice,95\nBob,70\nCarol,88\n"
with tempfile.NamedTemporaryFile("w", suffix=".csv", delete=False) as f:
f.write(data); inp = f.name
out = inp + "_out.csv"
n = filter_csv(inp, out, "score", "80", op="gt")
print(f"Kept {n} rows (score > 80)" # 2: Alice 95, Carol 88
os.unlink(inp); os.unlink(out)

```

Section 9: Concurrency & Performance (Q83–Q90)

Q83. Implement: thread-safe counter with `threading.Lock`.

```
import threading

class SafeCounter:
    def __init__(self):
        self._value = 0
        self._lock = threading.Lock()

    def increment(self):
        with self._lock:
            self._value += 1

    def decrement(self):
        with self._lock:
            self._value -= 1

    @property
    def value(self):
        with self._lock:
            return self._value

counter = SafeCounter()
N = 100_000

def worker():
    for _ in range(N):
        counter.increment()

threads = [threading.Thread(target=worker) for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()

print(f"Expected: {5*N} Got: {counter.value}") # Always 500000
```

Q84. Implement: producer-consumer with `queue.Queue`.

```
import queue, threading, time, random

def producer(q, items):
    for item in items:
        time.sleep(random.uniform(0, 0.02))
        q.put(item)
    print(f"[P] produced {item}")
    q.put(None) # sentinel

def consumer(q, worker_id, results):
    while True:
        item = q.get()
        if item is None:
            q.put(None) # pass sentinel to next consumer
            break
        results.append(item * 2)
        print(f"[W{worker_id}] processed {item}")
        q.task_done()
```



```

task_q, results = queue.Queue(maxsize=5), []

prod = threading.Thread(target=producer, args=(task_q, range(1,8)))
workers = [threading.Thread(target=consumer, args=(task_q, i, results))
            for i in range(2)]

prod.start()
for w in workers: w.start()
prod.join()
for w in workers: w.join()
print("Results:", sorted(results)) # [2,4,6,8,10,12,14]

```

Q85. Implement: parallel file processing with ProcessPoolExecutor.

```

from concurrent.futures import ProcessPoolExecutor, as_completed
from pathlib import Path
import tempfile, os

def count_words(filepath):
    # CPU-bound task
    return filepath, len(Path(filepath).read_text().split())

def process_parallel(file_list, max_workers=4):
    results = {}
    with ProcessPoolExecutor(max_workers=max_workers) as ex:
        futures = {ex.submit(count_words, f): f for f in file_list}
        for fut in as_completed(futures):
            try:
                path, count = fut.result()
                results[path] = count
            except Exception as e:
                print(f"Error: {e}")
    return results

with tempfile.TemporaryDirectory() as tmp:
    files = []
    for i in range(4):
        p = os.path.join(tmp, f"doc{i}.txt")
        Path(p).write_text("word " * (i+1) * 100)
        files.append(p)
    counts = process_parallel(files, max_workers=2)
    for path, count in counts.items():
        print(f"{Path(path).name}: {count} words")

```

Q86. Implement: async HTTP requests with asyncio.gather.

```

import asyncio

# Production: use aiohttp (pip install aiohttp)
# async def fetch(session, url):
#     async with session.get(url) as r:
#         return url, r.status, await r.text()
# async def main(urls):
#     async with aiohttp.ClientSession() as s:
#         return await asyncio.gather(*[fetch(s, u) for u in urls])

# Standalone demo (no network)
async def fake_fetch(url, delay=0.05):
    await asyncio.sleep(delay) # non-blocking sleep

```

```

return url, 200, 1024

async def fetch_all(urls):
    tasks = [fake_fetch(url) for url in urls]
    return await asyncio.gather(*tasks)

async def main():
    urls = [f"https://api.example.com/item/{i}" for i in range(5)]
    results = await fetch_all(urls)
    for url, status, size in results:
        print(f"{status} {size}B {url}")

asyncio.run(main())

```

Q87. Implement: asyncio semaphore to cap concurrent requests.

```

import asyncio

async def fetch_with_limit(sem, url):
    async with sem: # at most N concurrent
        await asyncio.sleep(0.05) # simulate HTTP
    return url, 200

async def main():
    urls = [f"https://api.example.com/{i}" for i in range(20)]
    sem = asyncio.Semaphore(5) # max 5 concurrent
    tasks = [fetch_with_limit(sem, url) for url in urls]
    results = await asyncio.gather(*tasks)
    ok = sum(1 for _, code in results if code == 200)
    print(f"Fetched {ok}/{len(urls)} successfully")

asyncio.run(main())

# Threading semaphore equivalent
import threading, time

sem = threading.Semaphore(3)
def worker(n):
    with sem:
        print(f"Worker {n} (max 3 concurrent)")
        time.sleep(0.05)

threads = [threading.Thread(target=worker, args=(i,)) for i in range(8)]
for t in threads: t.start()
for t in threads: t.join()

```

Q88. Implement: profile and benchmark slow vs fast function.

```

import time, cProfile, pstats, io

def find_common_slow(list1, list2):
    # O(n*m) nested loops
    return [x for x in list1 if x in list2]

def find_common_fast(list1, list2):
    # O(n+m) set intersection
    return list(set(list1) & set(list2))

n = 5000

```

```

l1 = list(range(n))
l2 = list(range(n//2, n + n//2))

t0 = time.perf_counter()
find_common_slow(l1, l2)
slow_ms = (time.perf_counter()-t0)*1000

t0 = time.perf_counter()
find_common_fast(l1, l2)
fast_ms = (time.perf_counter()-t0)*1000

speedup = slow_ms / max(fast_ms, 0.001)
print(f"Slow: {slow_ms:.1f}ms Fast: {fast_ms:.1f}ms Speedup: {speedup:.0f}x")

# Profile with cProfile
pr = cProfile.Profile()
pr.enable()
find_common_fast(l1, l2)
pr.disable()
stream = io.StringIO()
pstats.Stats(pr, stream=stream).sort_stats("cumulative").print_stats(3)
print(stream.getvalue())

```

Q89. Implement: batch processor with chunking and progress tracking.

```

import time
from typing import Callable, Iterable

def batch_process(items, processor: Callable, batch_size=100, verbose=True):
    items = list(items)
    total = len(items)
    results, errors = [], []

    for i in range(0, total, batch_size):
        batch = items[i:i+batch_size]
        for item in batch:
            try:
                results.append(processor(item))
            except Exception as e:
                errors.append((item, str(e)))
        if verbose:
            done = min(i+batch_size, total)
            pct = done/total*100
            print(f" {done}/{total} ({pct:.0f}%) | errors={len(errors)}")

    return results, errors

def process_item(n):
    if n % 7 == 0: raise ValueError(f"Bad: {n}")
    return n * n

results, errors = batch_process(range(1, 30), process_item, batch_size=10)
print(f"OK: {len(results)}, Errors: {len(errors)}")
print("Failed:", [e[0] for e in errors])

```

Q90. Implement: rate limiter decorator using token bucket.

```

import time, threading, functools

```

```
class RateLimiter:
    def __init__(self, calls_per_second):
        self.interval = 1.0 / calls_per_second
        self.last_call = 0.0
        self._lock = threading.Lock()

    def __call__(self, func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            with self._lock:
                now = time.perf_counter()
                elapsed = now - self.last_call
                if elapsed < self.interval:
                    time.sleep(self.interval - elapsed)
                self.last_call = time.perf_counter()
            return func(*args, **kwargs)
        return wrapper

@RateLimiter(calls_per_second=5) # max 5 calls/sec
def fetch_api(endpoint):
    return f"data from {endpoint}"

start = time.perf_counter()
for i in range(6):
    result = fetch_api(f"/item/{i}")
    elapsed = time.perf_counter() - start
    print(f"t={elapsed:.2f}s {result}")
# Output shows ~0.2s gaps between calls
```

Section 10: Unit Testing & CLI Automation (Q91–Q100)

Q91. Implement: mock external dependencies with unittest.mock.

```
from unittest.mock import patch, MagicMock
import unittest, json, urllib.request

# Code under test
def get_user(user_id):
    url = f"https://api.example.com/users/{user_id}"
    with urllib.request.urlopen(url) as r:
        return json.loads(r.read())

class TestGetUser(unittest.TestCase):

    @patch("urllib.request.urlopen")
    def test_success(self, mock_open):
        mock_response = MagicMock()
        mock_response.read.return_value = b'{"id":1,"name":"Alice"}'
        mock_response.__enter__ = lambda s: s
        mock_response.__exit__ = MagicMock(return_value=False)
        mock_open.return_value = mock_response

        result = get_user(1)
        self.assertEqual(result["name"], "Alice")
        mock_open.assert_called_once_with(
            "https://api.example.com/users/1")

    @patch("urllib.request.urlopen")
    def test_network_error(self, mock_open):
        mock_open.side_effect = ConnectionError("Down")
        with self.assertRaises(ConnectionError):
            get_user(999)

unittest.main(argv=[""], verbosity=2, exit=False)
```

Q92. Implement: parameterized tests without a framework.

```
import unittest

def is_palindrome(s):
    s = "".join(c.lower() for c in s if c.isalnum())
    return s == s[::-1]

class TestPalindrome(unittest.TestCase): pass

cases = [
    ("racecar", True),
    ("hello", False),
    ("A man a plan a canal Panama", True),
    ("", True),
    ("ab", False),
]

def make_test(s, expected):
    def test(self):
        self.assertEqual(is_palindrome(s), expected,
            f"is_palindrome({s!r}) should be {expected}")
```

```

return test

for i, (s, exp) in enumerate(cases):
    name = f"test_{i:02d}_{s[:10].replace(chr(32),chr(95))}"
    setattr(TestPalindrome, name, make_test(s, exp))

unittest.main(argv=[""], verbosity=2, exit=False)

```

Q93. Implement: integration test for a file transform pipeline.

```

import unittest, csv, tempfile, os
from pathlib import Path

def transform_csv(input_path, output_path, multiplier=2):
    with open(input_path) as fin, open(output_path, "w", newline="") as fout:
        reader = csv.DictReader(fin)
        writer = csv.DictWriter(fout, fieldnames=reader.fieldnames)
        writer.writeheader()
        for row in reader:
            try: row["value"] = str(float(row["value"]) * multiplier)
            except (ValueError, KeyError): pass
        writer.writerow(row)

class TestTransform(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.mkdtemp()
        self.inp = os.path.join(self.tmp, "in.csv")
        self.out = os.path.join(self.tmp, "out.csv")

    def tearDown(self):
        import shutil; shutil.rmtree(self.tmp)

    def _read_out(self):
        return list(csv.DictReader(open(self.out)))

    def test_doubles(self):
        Path(self.inp).write_text("name,value\nAlice,100\nBob,50\n")
        transform_csv(self.inp, self.out)
        rows = self._read_out()
        self.assertEqual(rows[0]["value"], "200.0")

    def test_non_numeric(self):
        Path(self.inp).write_text("name,value\nAlice,N/A\n")
        transform_csv(self.inp, self.out)
        self.assertEqual(self._read_out()[0]["value"], "N/A")

unittest.main(argv=[""], verbosity=2, exit=False)

```

Q94. Implement: test data factory for repeatable test setup.

```

from dataclasses import dataclass, field
from typing import List

@dataclass
class UserFactory:
    _counter: int = field(default=0, init=False, repr=False)

    def build(self, **overrides):
        self._counter += 1

```

```

n = self._counter
user = {
    "id": n,
    "name": f"TestUser{n}",
    "email": f"user{n}@example.com",
    "active": True,
    "roles": ["user"],
}
user.update(overrides)
return user

def build_admin(self, **overrides):
    return self.build(roles=["user", "admin"], **overrides)

def build_batch(self, count, **overrides):
    return [self.build(**overrides) for _ in range(count)]

factory = UserFactory()
user1 = factory.build()
admin = factory.build_admin(name="Alice Admin")
users = factory.build_batch(3, active=False)

print(user1)
print(admin)
for u in users:
    assert not u["active"], "Should be inactive"
print("Factory assertions passed!")

```

Q95. Implement: minimal test runner without a framework.

```

import traceback, time

class SimpleTestRunner:
    def __init__(self):
        self.tests = []
        self.passed = self.failed = 0

    def test(self, fn):
        self.tests.append(fn)
        return fn

    def run(self):
        t0 = time.perf_counter()
        for fn in self.tests:
            try:
                fn()
                self.passed += 1
                print(f" PASS {fn.__name__}")
            except AssertionError as e:
                self.failed += 1
                print(f" FAIL {fn.__name__}: {e}")
            except Exception as e:
                self.failed += 1
                print(f" ERROR {fn.__name__}: {e}")
        ms = (time.perf_counter()-t0)*1000
        print(f"\n{self.passed+self.failed} tests {ms:.1f}ms ",
              f"{self.passed} passed {self.failed} failed")

r = SimpleTestRunner()

```

```

@r.test
def test_arithmetic():
    assert 1 + 1 == 2
    assert 10 / 2 == 5.0

@r.test
def test_strings():
    assert "hello".upper() == "HELLO"
    assert " hi ".strip() == "hi"

@r.test
def test_lists():
    assert sorted([3,1,2]) == [1,2,3]
    assert max([1,5,3]) == 5

r.run()

```

Q96. Implement: CLI tool with argparse subcommands.

```

import argparse, json, csv, sys
from pathlib import Path

def cmd_count(args):
    path = Path(args.file)
    lines = path.read_text().splitlines()
    print(f"Lines: {len(lines)}")
    if args.words:
        print(f"Words: {sum(len(l.split()) for l in lines)}")

def cmd_convert(args):
    p = Path(args.input)
    if p.suffix != ".csv":
        print("Only CSV supported", file=sys.stderr); sys.exit(1)
    rows = list(csv.DictReader(p.open()))
    Path(args.output).write_text(json.dumps(rows, indent=2))
    print(f"Converted {len(rows)} rows")

def build_parser():
    parser = argparse.ArgumentParser(description="File utility")
    subs = parser.add_subparsers(dest="cmd", required=True)
    p_cnt = subs.add_parser("count", help="Count lines/words")
    p_cnt.add_argument("file")
    p_cnt.add_argument("-w", "--words", action="store_true")
    p_cnt.set_defaults(func=cmd_count)
    p_conv = subs.add_parser("convert", help="CSV -> JSON")
    p_conv.add_argument("input")
    p_conv.add_argument("output")
    p_conv.set_defaults(func=cmd_convert)
    return parser

import tempfile
with tempfile.NamedTemporaryFile("w", suffix=".txt", delete=False) as f:
    f.write("hello world\nfoo bar baz\n"); name=f.name
parser = build_parser()
args = parser.parse_args(["count", name, "-w"])
args.func(args) # Lines: 2 Words: 5
import os; os.unlink(name)

```


Q97. Implement: dependency injection for testable scripts.

```
from typing import Protocol

class Storage(Protocol):
    def read(self, key: str) -> str: ...
    def write(self, key: str, value: str) -> None: ...

class Notifier(Protocol):
    def notify(self, message: str) -> None: ...

# Business logic (injected)
class DataProcessor:
    def __init__(self, storage: Storage, notifier: Notifier):
        self.storage = storage
        self.notifier = notifier

    def process(self, key, data):
        self.storage.write(key, data.upper())
        self.notifier.notify(f"Processed {key}: {len(data)} chars")

# Fakes for tests
class FakeStorage:
    def __init__(self): self.data = {}
    def read(self, k): return self.data.get(k, "")
    def write(self, k, v): self.data[k] = v

class FakeNotifier:
    def __init__(self): self.msgs = []
    def notify(self, m): self.msgs.append(m)

storage = FakeStorage()
notifier = FakeNotifier()
proc = DataProcessor(storage, notifier)
proc.process("result.txt", "hello world")

assert storage.data["result.txt"] == "HELLO WORLD"
assert len(notifier.msgs) == 1
print("DI tests passed!")
```

Q98. Implement: health-check script with exit codes.

```
import sys, os, shutil

EXIT_OK = 0; EXIT_WARN = 1; EXIT_CRIT = 2; EXIT_UNK = 3

def check(name, fn, warn=None, crit=None):
    try:
        value, message = fn()
        if crit is not None and value >= crit:
            return "CRITICAL", name, message
        if warn is not None and value >= warn:
            return "WARNING", name, message
        return "OK", name, message
    except Exception as e:
        return "UNKNOWN", name, str(e)

def disk_used_pct(path="/"):
    total, used, free = shutil.disk_usage(path)
    pct = used/total*100
```

```

return pct, f"Disk: {pct:.1f}% ({free//1024//1024}MB free)"

def check_env_required():
    missing = [k for k in ["HOME", "PATH"] if not os.environ.get(k)]
    return len(missing), f"Missing env vars: {missing}" if missing else "All env vars OK"

checks = [
    check("disk", disk_used_pct, warn=80, crit=95),
    check("env", check_env_required),
]

worst = EXIT_OK
levels = {"OK":0, "WARNING":1, "CRITICAL":2, "UNKNOWN":3}
for status, name, msg in checks:
    worst = max(worst, levels[status])
print(f"[{status:8s}] {name}: {msg}")

print(f"\nExit code: {worst}")
sys.exit(worst)

```

Q99. Implement: complete ETL automation script (read, validate, transform, write).

```

import csv, json, logging, sys, os
from pathlib import Path

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)-8s %(message)s",
                    datefmt="%H:%M:%S", stream=sys.stdout)
log = logging.getLogger("etl")

def load_csv(path):
    with open(path, newline="") as f:
        return list(csv.DictReader(f))

def validate(rows):
    valid, errors = [], 0
    for row in rows:
        try:
            row["amount"] = float(row["amount"])
            if row["amount"] < 0: raise ValueError("Negative")
            valid.append(row)
        except (KeyError, ValueError) as e:
            log.warning(f" Skip row {row}: {e}")
            errors += 1
    return valid, errors

def transform(rows):
    from collections import defaultdict
    by_cat = defaultdict(list)
    for row in rows:
        by_cat[row.get("category", "misc")].append(row["amount"])
    return {
        "total": sum(r["amount"] for r in rows),
        "count": len(rows),
        "by_category": {k: {"sum": sum(v), "count": len(v)}}
        for k,v in by_cat.items()
    }

def run(inp_csv, out_json):

```

```

log.info(f"Loading {inp_csv}")
rows = load_csv(inp_csv)
log.info(f" {len(rows)} rows")
valid, errors = validate(rows)
if errors: log.warning(f" {errors} invalid rows")
summary = transform(valid)
log.info(f" Total={summary["total"]:.2f} Valid={summary["count"]}")
Path(out_json).write_text(json.dumps(summary, indent=2))
log.info(f"Report -> {out_json}")
return summary

import tempfile
data = "name,amount,category\nAlice,100,sales\nBob,bad,ops\nCarol,200,sales\n"
with tempfile.NamedTemporaryFile("w",suffix=".csv",delete=False) as f:
    f.write(data); inp=f.name
    out = inp+"_report.json"
    result = run(inp, out)
    print(json.dumps(result, indent=2))
    os.unlink(inp); os.unlink(out)

```

Q100. Implement: complete production-grade automation script template.

```

"""
Production Script Template – covers all CoderPad automation patterns.
"""

import argparse, logging, sys, os, json
from pathlib import Path
from dataclasses import dataclass
from typing import Optional

# 1. Configuration via dataclass
@dataclass
class Config:
    input_dir: Path
    output_dir: Path
    dry_run: bool = False
    verbose: bool = False

# 2. Logger setup
def setup_logger(verbose=False):
    level = logging.DEBUG if verbose else logging.INFO
    logging.basicConfig(level=level,
        format="%(asctime)s %(levelname)-8s %(message)s",
        datefmt="%H:%M:%S")
    return logging.getLogger("script")

# 3. Core business logic (testable, no I/O side effects)
def process_file(path: Path) -> dict:
    text = path.read_text()
    words = text.split()
    return {"file": path.name, "words": len(words), "chars": len(text)}

# 4. Orchestrator with error handling
def run(cfg: Config, log: logging.Logger) -> int:
    cfg.output_dir.mkdir(parents=True, exist_ok=True)
    files = list(cfg.input_dir.glob("*.txt"))
    log.info(f"Processing {len(files)} files from {cfg.input_dir}")
    results, errors = [], 0
    for f in files:

```

```

try:
    result = process_file(f)
    results.append(result)
    log.debug(f" OK: {f.name}")
    if not cfg.dry_run:
        out = cfg.output_dir / (f.stem + ".json")
        out.write_text(json.dumps(result, indent=2))
    except Exception as e:
        log.error(f" FAIL: {f.name}: {e}")
        errors += 1
    log.info(f"Done: {len(results)} ok, {errors} errors")
    return 0 if errors == 0 else 1

# 5. CLI entry point
def main() -> int:
    p = argparse.ArgumentParser(description="Process text files")
    p.add_argument("input_dir", type=Path)
    p.add_argument("output_dir", type=Path)
    p.add_argument("--dry-run", action="store_true")
    p.add_argument("-v", "--verbose", action="store_true")
    args = p.parse_args()
    cfg = Config(args.input_dir, args.output_dir, args.dry_run, args.verbose)
    log = setup_logger(cfg.verbose)
    return run(cfg, log)

# Demo (bypassing argparse)
import tempfile
with tempfile.TemporaryDirectory() as inp, tempfile.TemporaryDirectory() as out:
    for i in range(3):
        Path(inp, f"doc{i}.txt").write_text(f"hello world {i}\n" * 10)
    cfg = Config(Path(inp), Path(out), dry_run=False, verbose=True)
    log = setup_logger(verbose=True)
    exit_code = run(cfg, log)
    print("Exit code:", exit_code)

```